



Università degli Studi di Salerno

Dipartimento di Informatica

Corso di Ingegneria del Software: Tecniche Avanzate

CodeSmile



Post Modifications Testing
Versione 1.0

Team

Choaib Goumri NF22500223
Mattia Gallucci NF22500127

Repository GitHub

Anno Accademico 2025–2026

Indice

1	Introduzione	1
1.1	Ambiente di Test	1
2	Post-Modification Testing	2
2.1	CR Review & Analysis	2
2.1.1	CR01 - Stabilità e Accessibilità dell'Ambiente (Docker)	2
2.1.2	CR02 - Progress Bar CLI	3
2.1.3	CR03 - Smell: Randomness Uncontrolled	3
2.1.4	CR04 - Smell: Hyperparameter not explicitly set	4
2.1.5	CR05 - Metrica: "Densità degli Smell"	4
2.2	Regression Testing	5
2.2.1	Obiettivo	5
2.2.2	Strategia	5
2.2.3	Esecuzione Test Suite	5
2.2.4	Copertura del Codice (Code Coverage)	5
2.3	Unit Test	6
2.3.1	Unit Test con JEST	8
2.4	Integration Test	9
2.5	System Test	10
2.5.1	Testing End-to-End con Cypress	11

1 Introduzione

Il documento descrive le **attività di testing** eseguite a seguito dell'implementazione delle **change requests**. Le attività sono state finalizzate a **valutare l'impatto** delle modifiche introdotte e a garantire la qualità complessiva del sistema.

In particolare, gli **obiettivi** del testing sono stati i seguenti:

- **Post-Modification Testing:** verificare la corretta implementazione e il funzionamento delle nuove funzionalità e delle modifiche apportate al sistema.
- **Regression Testing:** accertare che le funzionalità preesistenti non abbiano subito alterazioni o regressioni a seguito delle modifiche implementate.

1.1 Ambiente di Test

Per garantire la riproducibilità dei risultati, i test sono stati eseguiti nel seguente ambiente controllato:

- **Sistema Operativo:** Windows 11
- **Interprete Python:** Versione 3.11
- **Gestione Dipendenze:** Librerie installate conformemente al file requirements.txt.
- **Repository:** https://github.com/ChoaibGoumri/smell_ai

2 Post-Modification Testing

2.1 CR Review & Analysis

Le attività di Post-Modification Testing hanno avuto come obiettivo la verifica delle componenti introdotte o modificate dalle cinque Change Requests (CR01-CR05) pianificate nell'Impact Analysis. L'obiettivo è stato confermare che le nuove funzionalità (Progress Bar, nuove regole di detection, metriche di densità) e le ristrutturazioni architetturali (Docker support) operino come previsto, rispettando i requisiti funzionali e non funzionali.

Tutti i test eseguiti sui confini delle modifiche hanno dato **esito positivo**, validando gli Actual Impact Set (AIS) definiti in fase di implementazione.

2.1.1 CR01 - Stabilità e Accessibilità dell'Ambiente (Docker)

Tipo di modifica: Correttiva

Componenti coinvolte:

Sono stati modificati 9 file totali, coinvolgendo entry point, router e utility per gestire l'agnosticismo dell'ambiente (Local vs Container), secondo l'AIS rilevato:

- **Gateway:** `webapp/gateway/main.py`
- **AI Service:** `app/main.py`, `routers/detect_smell.py`, `utils/model.py`
- **Static Analysis Service:** `app/main.py`, `routers/detect_smell.py`, `utils/static_analysis.py`
- **Report Service:** `app/main.py`, `routers/report.py`

Verifiche effettuate:

- **Environment Variable Injection:** Verifica del corretto caricamento degli endpoint dinamici tramite `docker-compose`.
- **Conditional Imports:** Test di avvio sia in ambiente locale (Python venv) che containerizzato per confermare che le clausole `try-except` sugli import risolvano correttamente i percorsi.
- **Inter-Service Communication:** Verifica della raggiungibilità dei microservizi dal Gateway utilizzando i nomi DNS interni di Docker.

Esito: Funzionalità verificata. Il sistema si avvia correttamente in entrambi gli scenari senza modifiche manuali al codice sorgente.

2.1.2 CR02 - Progress Bar CLI

Tipo di modifica: Perfettiva

Componenti coinvolte:

Sono stati coinvolti il core dell'analisi e la gestione delle dipendenze:

- `components/project_analyzer.py`
- `requirements.txt` (aggiunta libreria per progress bar)

Verifiche effettuate:

- **Sync Test:** Verifica della sincronizzazione tra l'avanzamento della barra e il numero effettivo di file processati dall'Inspector.
- **Visual Feedback:** Validazione del rendering testuale della barra su terminali standard durante l'analisi di progetti di grandi dimensioni.
- **Overhead Test:** Verifica che l'aggiornamento della UI non introduca ritardi significativi nel tempo totale di analisi.

Esito: Funzionalità conforme. La CLI fornisce feedback in tempo reale migliorando la User Experience.

2.1.3 CR03 - Smell: Randomness Uncontrolled

Tipo di modifica: Evolutiva

Componenti coinvolte:

È stato introdotto un nuovo modulo di detection e aggiornato il checker:

- `components/rule_checker.py`
- `detection_rules/generic/randomness_uncontrolled.py` (Nuovo file)

Verifiche effettuate:

- **Positive Detection:** Test su snippet di codice ML contenenti funzioni come `train_test_split` o modelli (es. `RandomForest`) senza parametro `random_state/seed`.
- **Negative Detection:** Verifica che lo smell *non* venga segnalato se il seed è impostato esplicitamente.
- **AST Navigation:** Verifica della corretta identificazione delle chiamate di funzione sensibili alla casualità.

Esito: La regola identifica correttamente le violazioni di riproducibilità.

2.1.4 CR04 - Smell: Hyperparameter not explicitly set

Tipo di modifica: Perfettiva

Componenti coinvolte:

Oltre alla nuova regola, è stato necessario intervenire sull'estrattore (DIS):

- `obj_dictionaries/models.csv`
- `detection_rules/generic/hyperparameters_not_explicitly_set.py`
- `code_extractor/model_extractor.py`

Verifiche effettuate:

- **Default Values:** Test su istanziazioni di modelli vuote per verificare la segnalazione dello smell.
- **Config Validation:** Verifica che la regola legga correttamente i parametri obbligatori dal file CSV di configurazione.
- **False Positives:** Verifica che l'uso di parametri non critici non faccia scattare l'allarme.

Esito: La regola previene il rischio di "Silent Failure" dovuto ai default delle librerie, come da requisiti.

2.1.5 CR05 - Metrica: "Densità degli Smell"

Tipo di modifica: Evolutiva

Componenti coinvolte:

Modifica trasversale (Cross-cutting) che ha interessato Frontend, Backend e Reporting:

- **Frontend:** `webapp/app/upload-python/page.tsx`, `webapp/types/types.ts`, `webapp/utils/api.ts`
- **Core Analysis:** `components/inspector.py`, `components/project_analyzer.py`
- **Reporting:** `report/report_generator.py`
- **Backend Services:** `routers/detect_smell.py`, `schemas/responses.py`, `utils/-static_analysis.py`

Verifiche effettuate:

- **LOC Calculation:** Verifica dell'accuratezza del conteggio delle righe di codice (escludendo commenti/linee vuote se previsto).
- **Formula Test:** Verifica matematica del calcolo $\frac{\text{Smells}}{\text{LOC}}$.
- **Report Integration:** Controllo della presenza della nuova colonna "Density" nei report CSV/Excel generati e nel JSON di risposta verso il Frontend.
- **UI Rendering:** Verifica della visualizzazione del dato di densità nella dashboard dei risultati.

Esito: La metrica è correttamente calcolata e propagata attraverso l'intera pipeline, dal parsing iniziale alla visualizzazione finale.

2.2 Regression Testing

2.2.1 Obiettivo

Garantire che le modifiche introdotte dalle CR (CR01-CR05) non abbiano alterato il comportamento del sistema nelle funzionalità preesistenti, mantenendo la compatibilità con i flussi di lavoro storici.

2.2.2 Strategia

La regressione è stata condotta tramite la riesecuzione completa della suite di test automatizzati e il confronto degli output generati con gli oracoli attesi.

2.2.3 Esecuzione Test Suite

Test automatici

È stata eseguita una suite completa di test di regressione sull'intera codebase.

Stato complessivo: SUPERATO

Categoria di test	Tipo	Totale	Superati	Falliti	Tasso di successo
Unit Tests	CLI Components	16	16	0	100%
Unit Tests	Code Extractors	43	43	0	100%
Unit Tests	Core Components	57	57	0	100%
Unit Tests	Detection Rules (API)	27	27	0	100%
Unit Tests	Detection Rules (Generic)	78	78	0	100%
Unit Tests	GUI Components	55	55	0	100%
Unit Tests	Report Generation	44	44	0	100%
Integration Tests	Component Integration	7	7	0	100%
Integration Tests	Workflow Integration	1	1	0	100%
System Tests	Full System Test	24	24	0	100%
Web App	Jest	27	27	0	100%
Web App	Cypress	27	27	0	100%
Totale		312	312	0	100%

Tabella 2.1: Riepilogo esecuzione Test Suite

2.2.4 Copertura del Codice (Code Coverage)

L'analisi della copertura del codice ha evidenziato i seguenti risultati:

- **Copertura Complessiva del Progetto: 90%.**
- **Copertura Modifiche (New/Modified Files): 100%.**

I file oggetto di modifica o creazione durante le Change Request (CR01-CR05) sono stati coperti integralmente dai nuovi casi di test introdotti, garantendo la massima affidabilità delle nuove feature. Il delta del 10% sulla copertura globale è relativo a componenti legacy o di interfaccia grafica non impattati dalle modifiche correnti.

2.3 Unit Test

Di seguito viene riportata in dettaglio la tabella dei vari Unit Test eseguiti per validare le Change Request.

Tipo	ID	Funzione/Classe	Input	Oracolo Atteso
UT	CR01-01	gateway/main.py (URL configuration)	Variabili d'ambiente per URL servizi (AISERVICE_URL, STATIC_URL, REPORT_URL)	URL caricati correttamente, chiamate HTTP indirizzate ai servizi corretti
UT	CR01-02	aiservice/app/main.py (import fix)	Import router con path assoluto/relativo	Import risolto senza ModuleNotFoundError
UT	CR01-03	staticanalysis/app/main.py (import fix)	Import router con path assoluto/relativo	Import risolto senza ModuleNotFoundError
UT	CR01-04	report/app/main.py (import fix)	Import router con path assoluto/relativo	Import risolto senza ModuleNotFoundError
UT	CR02-01	CodeSmileCLI.execute	Args con input/output validi, parallel=False	analyze_project() chiamato, "Analysis completed" stampato
UT	CR02-02	CodeSmileCLI.execute	Args con parallel=True, multiple=True, max_walkers=5	analyze_projects_parallel() chiamato con 5 workers
UT	CR02-03	CodeSmileCLI.execute	Args con max_walkers=-1 (non valido)	ValueError "max_walkers must be greater than 0."
UT	CR02-04	CodeSmileCLI.execute	Args con resume=True	clean_output_directory() NON chiamato
UT	CR02-05	CodeSmileCLI.execute	Args senza input path	SystemExit, "Error: Please specify both input and output folders."
UT	CR02-06	CodeSmileCLI.print_configuration	Args con configurazione valida	Configurazione stampata correttamente in console
UT	CR03-01	ChainIndexing-Smell.detect	Codice con df['a'][0] (chain indexing)	1 smell rilevato, "Chain_Indexing", "Chained indexing detected", line=4
UT	CR03-02	ChainIndexing-Smell.detect	Codice con df.loc[0, 'a'] (accesso corretto)	0 smell rilevati
UT	CR03-03	ChainIndexing-Smell.detect	Codice senza pandas importato	0 smell rilevati (nessuna libreria pandas)
UT	CR03-04	ChainIndexing-Smell.detect	Codice con multipli chain indexing (df['a'][0] e df['a'][1])	2 smell rilevati alle righe corrette
UT	CR03-05	MatrixMultiplication-APIMisused.detect	Codice con np.matmul() (metodo corretto)	0 smell rilevati
UT	CR03-06	MatrixMultiplication-APIMisused.detect	Codice con np.dot() per moltiplicazione matrici	1 smell rilevato, "matrix_multiplication_API_misused"
UT	CR03-07	MatrixMultiplication-APIMisused.detect	Codice senza numpy importato	0 smell rilevati
UT	CR03-08	MatrixMultiplication-APIMisused.detect	Codice con multipli np.dot()	Multipli smell rilevati

Tipo	ID	Funzione/Classe	Input	Oracolo Atteso
UT	CR03-09	DataFrameConversion-APIMisused.detect	Codice con <code>df.to_numpy()</code> (metodo corretto)	0 smell rilevati
UT	CR03-10	DataFrameConversion-APIMisused.detect	Codice con <code>df.values</code> (metodo deprecato)	1 smell rilevato, "dataframe_conversion_API_misused"
UT	CR04-01	Hyperparameters-NotExplicitlySet-Smell.detect	<code>RandomForestClassifier()</code> senza <code>n_estimators</code>	1 smell rilevato, "Missing critical configurations: <code>n_estimators</code> ", line=3
UT	CR04-02	Hyperparameters-NotExplicitlySet-Smell.detect	<code>RandomForestClassifier(n_estimators=100)</code>	0 smell rilevati
UT	CR04-03	Hyperparameters-NotExplicitlySet-Smell.detect	<code>RandomForestClassifier(max_depth=5)</code> senza <code>n_estimators</code>	1 smell rilevato per <code>n_estimators</code> mancante
UT	CR04-04	Hyperparameters-NotExplicitlySet-Smell.detect	<code>KMeans()</code> senza <code>n_clusters</code>	1 smell rilevato, "Missing critical configurations: <code>n_clusters</code> "
UT	CR04-05	Hyperparameters-NotExplicitlySet-Smell.detect	Codice senza libreria importata	0 smell rilevati (nessuna libreria da verificare)
UT	CR04-06	Hyperparameters-NotExplicitlySet-Smell.detect	Import con alias (<code>sklearn.ensemble as se</code>)	Smell rilevati correttamente tramite risoluzione alias
UT	CR04-07	Hyperparameters-NotExplicitlySet-Smell.detect	Modello non presente in <code>models.csv</code>	0 smell rilevati (modello sconosciuto ignorato)
UT	CR04-08	Hyperparameters-NotExplicitlySet-Smell.detect	Parametri critici passati come posizionali	Smell rilevati solo se non specificati per nome
UT	CR04-09	Hyperparameters-NotExplicitlySet-Smell.get_full_function_name	Nodo AST con alias non in dizionario	Nome funzione diretto restituito (fallback)
UT	CR04-10	Hyperparameters-NotExplicitlySet-Smell.get_full_function_name	Nodo AST non Name né Attribute	None restituito senza crash
UT	CR05-01	DataFrameExtractor.extract_functions	Codice Python con 5 righe	LOC=5 calcolato correttamente
UT	CR05-02	Inspector.inspect	File Python con codice valido	Tupla (DataFrame smells, LOC) ritornata
UT	CR05-03	ReportGenerator.summary_report	DataFrame: filename="file1.py", smells=2, loc=1000	Density=0.002, Quality="Low" (< 0.005)
UT	CR05-04	ReportGenerator.summary_report	DataFrame: filename="file2.py", smells=1, loc=20	Density=0.05, Quality="High" (>= 0.05)
UT	CR05-05	ReportGenerator.summary_report	DataFrame con density=0.025 (tra 0.005 e 0.05)	Quality="Medium"
UT	CR05-06	ReportGenerator.summary_report	DataFrame con più file e progetti	Excel con sheet "proj_Files", colonne: filename, smells, loc, density, quality

Tipo	ID	Funzione/Classe	Input	Oracolo Atteso
UT	CR05-07	ReportGenerator.summary_report	DataFrame con LOC=0	Density gestita senza divisione per zero
UT	CR05-08	ReportGenerator.load_data	Path con file CSV validi	DataFrame caricato con dati corretti
UT	CR05-09	ReportGenerator.find_project_details	Directory con project_details folder	Lista file CSV trovata
UT	CR05-10	ReportGenerator.classify_density	Density < 0.005	Quality="Low"
UT	CR05-11	ReportGenerator.classify_density	Density >= 0.05	Quality="High"

2.3.1 Unit Test con JEST

È stato implementato un test con Jest nel file `webapp/tests/upload-python/page.test.tsx` per verificare il corretto funzionamento del componente Dashboard Metriche.

Il test simula una risposta positiva del servizio di analisi, restituendo i seguenti valori:

LOC = 100, Smell Density = 0.05 e Quality Rating = **High**. Successivamente, viene renderizzata la pagina `UploadPythonPage`, simulato il caricamento di un file Python e il click sul pulsante `Upload Code`.

Infine, il test verifica la visualizzazione della sezione `Code Quality Metrics`, la corretta visualizzazione del valore LOC, la formattazione della densità con quattro cifre decimali (0.0500) e l'etichetta del badge di qualità.

2.4 Integration Test

Di seguito viene riportata in dettaglio la tabella dei vari Integration Test eseguiti per verificare il corretto funzionamento tra le varie componenti.

Tipo	ID	Funzione/Classe	Input	Oracolo Atteso
IT	CR02-01	CLI → ProjectAnalyzer	Args mock con input/output path, chiamata execute()	ProjectAnalyzer istanziato e analyze_project() chiamato con input corretto
IT	CR02-02	GUI → ProjectAnalyzer	GUI con input/output path configurati, chiamata run_analysis()	ProjectAnalyzer.analyze_project() eseguito con parametri corretti
IT	CR02-03	ProjectAnalyzer → Inspector	Cartella con 2 file Python (.py)	Inspector.inspect() chiamato 2 volte, una per file
IT	CR02-04	Full Integration CLI → Analyzer → Inspector → RuleChecker	File Python in input, esecuzione CLI completa	File overview.csv creato in output/output/ con smell rilevati
IT	CR02-05	Full Integration GUI → Analyzer → Inspector → RuleChecker	File Python in input, esecuzione GUI completa	File overview.csv creato con smell, GUI non crasha
IT	CR03-01	Inspector → RuleChecker	File Python, Inspector chiama RuleChecker.rule_check()	RuleChecker.rule_check() invocato con AST e extracted_data corretti
IT	CR04-01	Inspector + CR4 Detection (Hyperparameters)	File con RandomForestClassifier() e KMeans() senza parametri critici	2 smell "hyperparameters_not_explicitly_set" rilevati nel DataFrame
IT	CR05-01	Density Full Flow (Analyzer → ReportGenerator)	Progetto con file Python, analisi + generazione report	Metrics CSV con LOC, Summary Excel con density e quality

2.5 System Test

La suite completa dei test di sistema, costituita da 20 scenari precedentemente definiti, è stata completamente validata ottenendo esito positivo.

A seguito dell'implementazione delle modifiche, si è reso necessario estendere il framework di testing mediante l'introduzione di un'ulteriore categoria all'interno dei parametri di configurazione basati sul metodo category partition.

I test case di seguito riportati sono stati progettati specificatamente per validare le nuove funzionalità introdotte.

Per le modifiche effettuate è stata aggiunta una nuova categoria ai parametri di configurazione del tool del category partition specificato:

Categoria	Scelte
Livello di Densità / Qualità (LD)	1. Low (Density < 0.5%) 2. Medium (0.5% ≤ Density < 5%) 3. High (Density ≥ 5%)

Tabella 2.4: Nuova categoria per System Test

Di seguito sono elencati i nuovi test case legati alla funzionalità aggiunta.

Test Case ID	Test Frame	Oracolo Atteso
TC_21	NF3, EF1, NP1, SP1, NCS3, TCS1, ME2, EP2, NW3, RES2, OUT1, DB0, EG0, LD1	Qualità Alta (Low Density): Il report Excel riporta density < 0.005 e quality = "Low".
TC_22	NF3, EF1, NP1, SP1, NCS3, TCS1, ME2, EP2, NW3, RES2, OUT1, DB0, EG0, LD2	Qualità Media (Medium Density): Il report Excel riporta 0.005 ≤ density < 0.05 e quality = "Medium".
TC_23	NF3, EF1, NP1, SP1, NCS3, TCS1, ME2, EP2, NW3, RES2, OUT1, DB0, EG0, LD3	Qualità Bassa (High Density): Il report Excel riporta density ≥ 0.05 e quality = "High".
TC_24	NF2, EF1, NP1, SP1, NCS2, TCS1, ME1, EP2, NW0, RES0, OUT1, RB1, EG1, LD3	WebApp Dashboard: La risposta API contiene density e quality popolati; la UI mostra il badge "High".

2.5.1 Testing End-to-End con Cypress

Per garantire la qualità e l'affidabilità dell'applicazione web *CodeSmile*, è stata implementata una suite di test end-to-end (E2E) utilizzando **Cypress** versione 13.17.0. La strategia di testing prevede **27 test** distribuiti in 7 file di specifica, progettati per coprire in modo sistematico i principali flussi utente dell'applicazione.

I test validano le seguenti aree funzionali:

- Navigazione della homepage e routing tra le sezioni dell'applicazione (`homepage.spec.cy.ts`, 5 test);
- Upload e analisi di file Python singoli in modalità statica e AI-based (`upload-python.spec.cy.ts`, 8 test);
- Gestione di progetti multi-file (`upload-project.spec.cy.ts`, 6 test);
- Generazione e visualizzazione dei report analitici (`report.spec.cy.ts`, 5 test).

Particolare attenzione è stata dedicata al rilevamento dei *code smell*, con test specifici per le metriche di qualità del codice, *Hyperparameter Smell* e *Randomness Uncontrolled Smell*.